

Azure Synapse Analytics Notes

Component / Concept	Core Function & Mechanics	Pricing & Compute Model	Primary Use Cases
Dedicated SQL Pools	Massively Parallel Processing (MPP) enterprise relational data warehouse	Provisioned Data Warehouse Units scaled on demand	Enterprise BI reporting, high-concurrency SQL queries, relational warehousing
Serverless SQL Pools	On-demand distributed query engine over data lake files	Billed per TB of data scanned	Ad-hoc file exploration, logical views over data lake, lightweight ELT
Synapse Spark Pools	Managed Apache Spark runtime integrated within Synapse Studio	Pay-as-you-go based on vCPU/RAM per minute	Data science, PySpark transformations, machine learning, streaming
Synapse Pipelines	Built-in ETL/ELT orchestration engine (powered by ADF)	Charged per activity execution & Data Movement Units	Hybrid data ingestion, workflow orchestration, scheduling
ADLS Gen2 Storage	Primary underlying data lake with Hierarchical Namespace	Azure Object Storage pricing (Hot / Cool / Archive tiers)	Storing raw files (CSV, Parquet, JSON) and Gold Delta Lake formats

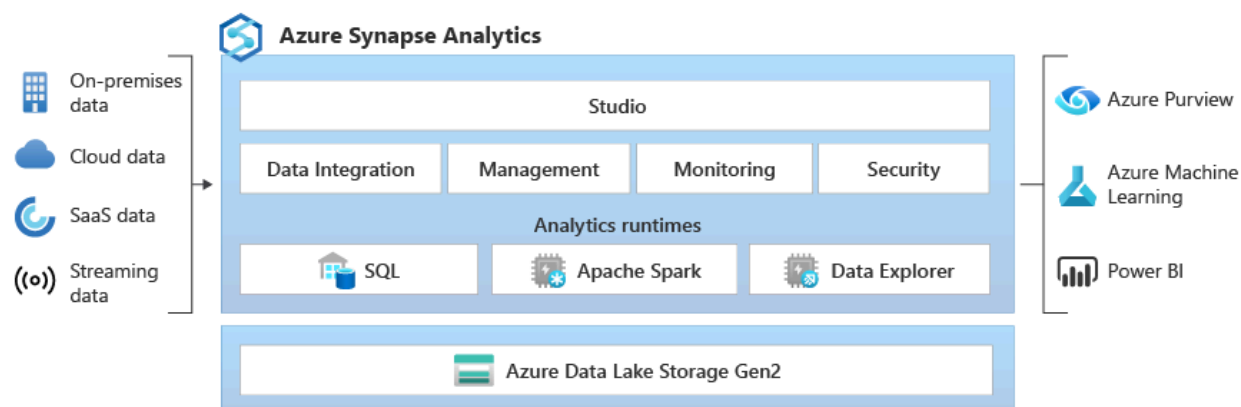
1. Introduction to Azure Synapse Analytics

What is Azure Synapse Analytics?

Azure Synapse Analytics is an integrated analytics platform that brings together big data processing, enterprise data warehousing, and data integration into a single interface called **Synapse Studio**.

Traditionally, organizations had to purchase and manage separate systems for ETL pipelines

(such as Azure Data Factory), data warehousing (such as SQL Server or Teradata), and big data processing (such as Apache Spark). Synapse unifies all of these capabilities into a single service.

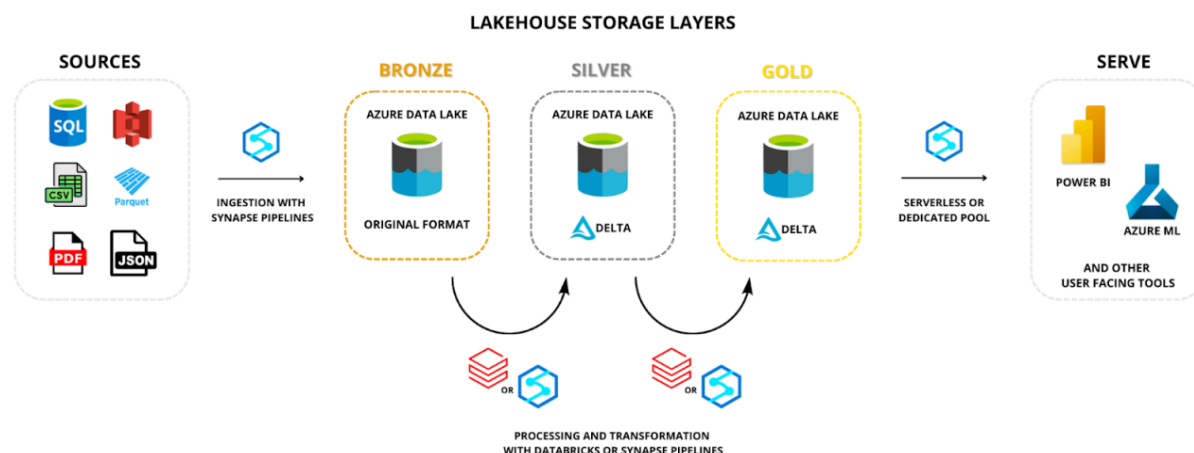


Key Workspace Components

1. **Synapse Studio:** A single web UI where developers write T-SQL, build PySpark notebooks, monitor job runs, and construct ETL pipelines.
2. **Synapse Pipelines:** Built directly on the Azure Data Factory (ADF) engine, providing data integration without needing an external ADF workspace.
3. **Primary ADLS Gen2 Account:** Every Synapse workspace is linked to a default Azure Data Lake Storage Gen2 account where workspace metadata and temporary logs are stored.

Medallion Architecture in Synapse

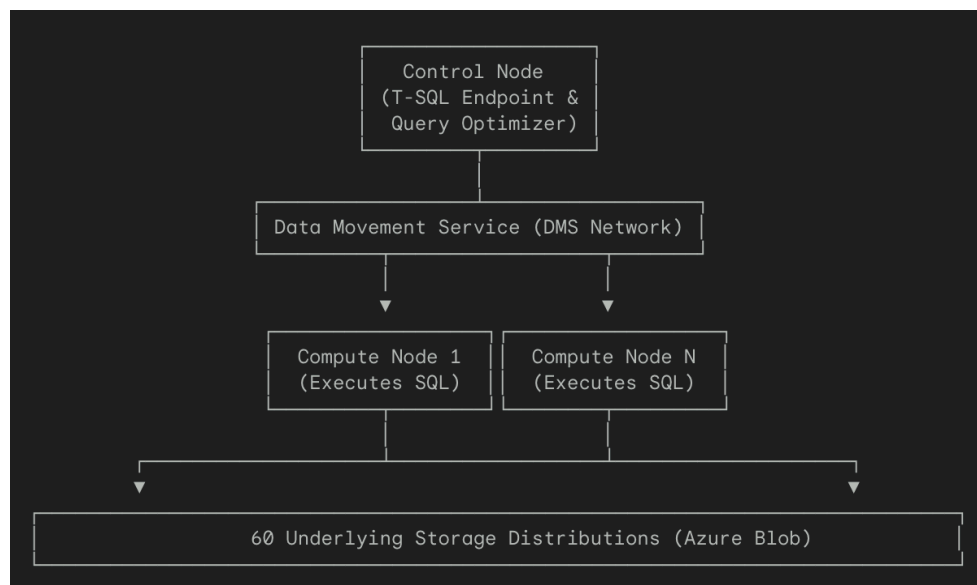
Synapse seamlessly implements the **Medallion Architecture** pattern across storage and compute layers:



- **Bronze Layer (Raw):** Landing zone for raw incoming source data (CSV, JSON) stored directly in ADLS Gen2.
- **Silver Layer (Cleansed):** Data is cleansed, validated, and converted into optimized file formats (Parquet or Delta Lake) using Synapse Spark or Serverless SQL.
- **Gold Layer (Curated):** High-performance curated tables loaded into Synapse Dedicated SQL Pools or represented as logical SQL views for business intelligence tools like Power BI.

2. Azure Synapse Architecture Deep Dive

Synapse Dedicated SQL Pools run on a **Massively Parallel Processing (MPP)** architecture. Compute resources are decoupled from storage, allowing you to scale or pause compute independently.



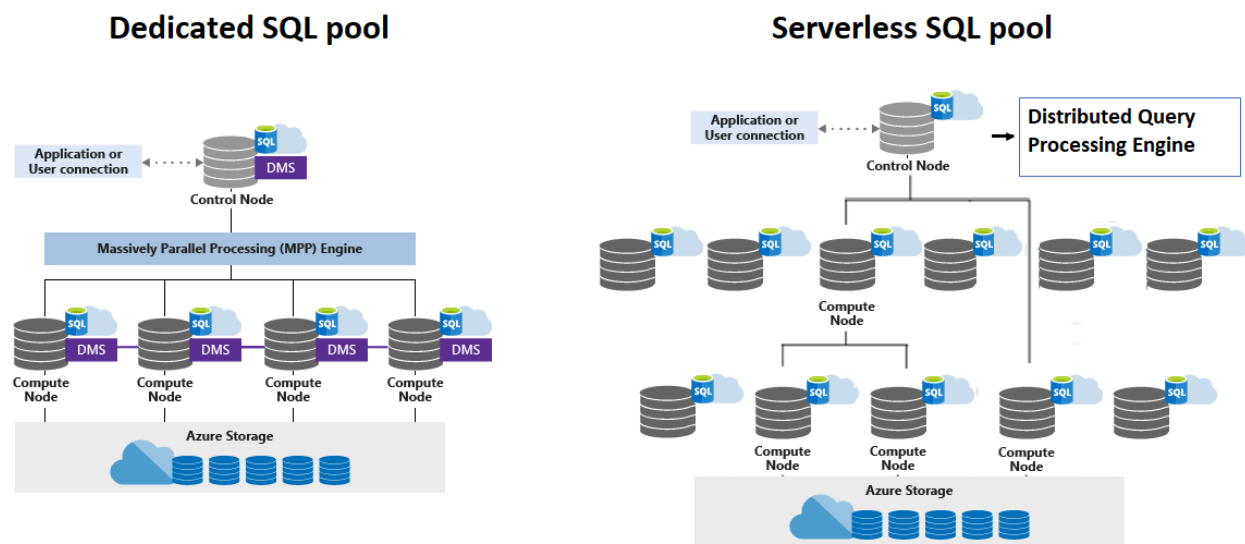
Core Architecture Components

1. **Control Node:**
 - Acts as the front-end interface accepting all client T-SQL requests.
 - Parses, optimizes, and translates user SQL queries into parallel execution instructions sent to individual Compute Nodes.
 - Applications connect directly to the Control Node endpoint.
2. **Compute Nodes:**
 - Worker engines that process data in parallel.
 - As you scale up **Data Warehouse Units**, Synapse allocates more Compute Nodes to distribute execution threads.
 - Scaling increases worker capacity without altering underlying storage.
3. **Data Movement Service (DMS):**
 - An internal high-speed transport network that moves data across compute nodes during query execution.

- DMS coordinates data shuffles required when joining or aggregating distributed tables.
4. **60 Storage Distributions:**
- Behind the scenes, data in every Dedicated SQL Pool is divided across exactly **60 fixed storage locations (distributions)**.
 - Regardless of your assigned DWU level, data remains stored across these 60 distributions. Compute nodes simply reassign ownership of these 60 distributions dynamically as compute scales up or down.

3. Synapse SQL Pools & Engine Types

Synapse offers three primary compute engine choices depending on workload requirements:



1. Dedicated SQL Pools

- **Compute Model:** Provisioned resources measured in Data Warehouse Units .
- **Storage Model:** Managed internal relational storage mapped across 60 distributions.
- **Best Used For:** Enterprise data warehousing, highly predictable analytical reporting, complex dimensional models, and high-concurrency workloads.

2. Serverless SQL Pools

- **Compute Model:** On-demand, auto-scaling compute requiring no cluster provisioning or management.
- **Storage Model:** Unmanaged cloud storage (directly reads files residing in ADLS Gen2).
- **Billing Model:** Billed directly per Terabyte of data processed by queries.
- **Best Used For:** Rapid file exploration, creating logical abstraction layers (views) on top of

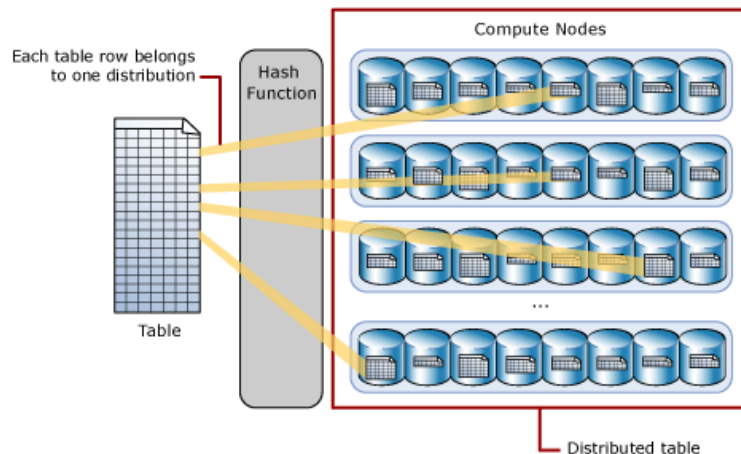
lake files, and lightweight data transformation.

3. Synapse Spark Pools

- **Compute Model:** Fully managed, auto-scaling Apache Spark cluster runtime.
- **Best Used For:** Data engineering with PySpark, Scala, or Spark SQL, handling unstructured/semi-structured data, machine learning, and stream processing.

4. Distribution Strategies for Dedicated SQL Pools

To optimize performance and minimize data movement across compute nodes during execution, table structures in Dedicated SQL Pools must be configured using one of three distribution strategies:



1. Hash Distribution (HASH)

- **Mechanics:** Applies a deterministic hash algorithm to a specified distribution column to assign each row to one of the 60 distributions.
- **Best For:** Large Fact Tables that are frequently joined or aggregated on a common key (e.g., CustomerKey or ProductKey).
- **Design Rule:** Choose a column with high cardinality (many unique values) that does not contain skewed data distributions to prevent uneven worker loads.

```
CREATE TABLE dbo.FactSales
(  
    SalesOrderNumber VARCHAR(50) NOT NULL,  
    CustomerKey INT NOT NULL,  
    OrderDateKey INT NOT NULL,  
    TotalAmount DECIMAL(18,2) NOT NULL  
)
```

```

WITH
(
    CLUSTERED COLUMNSTORE INDEX,
    DISTRIBUTION = HASH(CustomerKey)
);

```

2. Round-Robin Distribution (ROUND_ROBIN)

- **Mechanics:** Distributes incoming rows sequentially and randomly across all 60 distributions without analyzing data values.
- **Best For:** Temporary staging tables or quick data loading where join patterns are not yet known.
- **Trade-off:** Fast loading performance, but query joins may require data shuffles across compute nodes.

```

CREATE TABLE dbo.StagingSales
(
    SalesOrderNumber VARCHAR(50),
    CustomerKey INT,
    TotalAmount DECIMAL(18,2)
)
WITH
(
    HEAP,
    DISTRIBUTION = ROUND_ROBIN
);

```

3. Replicated Tables (REPLICATE)

- **Mechanics:** Copies the entire table onto every active Compute Node.
- **Best For:** Small Dimension Tables that are frequently joined to large Hash-Distributed fact tables.
- **Advantage:** Eliminates Data Movement Service (DMS) shuffles during queries, as every compute node holds the full dimension dataset locally.

```

CREATE TABLE dbo.DimProductCategory
(
    ProductCategoryKey INT NOT NULL,
    CategoryName VARCHAR(100) NOT NULL
)
WITH
(
    CLUSTERED INDEX (ProductCategoryKey),

```

```
DISTRIBUTION = REPLICATE  
);
```

Distribution Decision Framework

Table Type	Recommended Size	Distribution Strategy	Primary Rationale
Fact Table	>200gb	HASH(DistributionColumn)	Optimizes large joins and aggregations on key columns
Dimension Table	<2gb	REPLICATE	Bypasses data shuffling across compute nodes
Dimension Table	>2gb	HASH(PrimaryKey)	Prevents excessive memory overhead from replication
Staging / Raw Load	Any Size	ROUND_ROBIN	Maximizes raw data ingestion throughput

5. Serverless SQL Pools & ADLS Gen2 Data Ingestion

Synapse Serverless SQL Pools allow you to query files in ADLS Gen2 directly using standard T-SQL syntax without spinning up or paying for relational database instances.

Querying Data Lake Files via OPENROWSET

The OPENROWSET function allows SQL queries to read content directly from CSV, Parquet, or Delta files.

```
-- Querying Parquet files directly from ADLS Gen2  
SELECT  
    r.filepath(1) AS [Year],  
    r.filepath(2) AS [Month],  
    COUNT(*) AS TotalOrders,  
    SUM(CustomerAmount) AS TotalRevenue  
FROM
```

```

OPENROWSET(
    BULK 'https://stsynapseadls.dfs.core.windows.net/gold/sales/year=*/month=*/*.parquet',
    FORMAT = 'PARQUET'
) AS r
GROUP BY
    r.filepath(1),
    r.filepath(2);

```

Authentication & Access Setup

To allow Serverless SQL Pools to read securely from private ADLS Gen2 containers, you configure Database Scoped Credentials, Managed Identities, and External Connections.

Step 1: Create Database Scoped Credential using Workspace Managed Identity

```

CREATE DATABASE SCOPED CREDENTIAL [StorageManagedIdentityCredential]
WITH
    IDENTITY = 'Managed Identity';

```

Step 2: Create External Data Source

An **External Data Source** abstracts the physical storage account path and points to your storage container.

```

CREATE EXTERNAL DATA SOURCE [GoldDataLakeSource]
WITH (
    LOCATION = 'https://stsynapseadls.dfs.core.windows.net/gold',
    CREDENTIAL = [StorageManagedIdentityCredential]
);

```

Step 3: Create External File Formats

File formats inform Synapse how raw storage bytes should be parsed (e.g., CSV delimiter settings, compression algorithms).

```

-- Parquet File Format
CREATE EXTERNAL FILE FORMAT [ParquetFormat]
WITH (
    FORMAT_TYPE = PARQUET,
    DATA_COMPRESSION = 'org.apache.hadoop.io.compress.SnappyCodec'
);

```

```

-- CSV File Format

```



```

CREATE EXTERNAL FILE FORMAT [CsvFormat]
WITH (
    FORMAT_TYPE = DELIMITEDTEXT,
    FORMAT_OPTIONS (
        FIELD_TERMINATOR = ',',
        STRING_DELIMITER = '"',
        FIRST_ROW = 2,
        USE_TYPE_DEFAULT = FALSE
    )
);

```

Step 4: Create External Tables

External tables link SQL metadata directly to underlying storage files. Querying an external table reads live data from disk without duplicating storage inside a relational database engine.

```

CREATE EXTERNAL TABLE dbo.ExtSalesGold (
    SalesOrderID INT,
    OrderDate DATE,
    CustomerID INT,
    TotalDue DECIMAL(18,2)
)
WITH (
    LOCATION = '/sales/*.parquet',
    DATA_SOURCE = [GoldDataLakeSource],
    FILE_FORMAT = [ParquetFormat]
);

```

Transforming & Exporting Data via CETAS

CETAS (**Create External Table As Select**) reads data, transforms it via SQL, and writes the output as Parquet files into ADLS Gen2 while registering a queryable SQL table simultaneously.

```

CREATE EXTERNAL TABLE dbo.ExtCustomerSummary
WITH (
    LOCATION = '/exports/customer_summary/',
    DATA_SOURCE = [GoldDataLakeSource],
    FILE_FORMAT = [ParquetFormat]
)
AS
SELECT
    CustomerID,
    COUNT(SalesOrderID) AS TotalOrders,

```

```
SUM(TotalDue) AS LifetimeSpend
FROM
    dbo.ExtSalesGold
GROUP BY
    CustomerID;
```

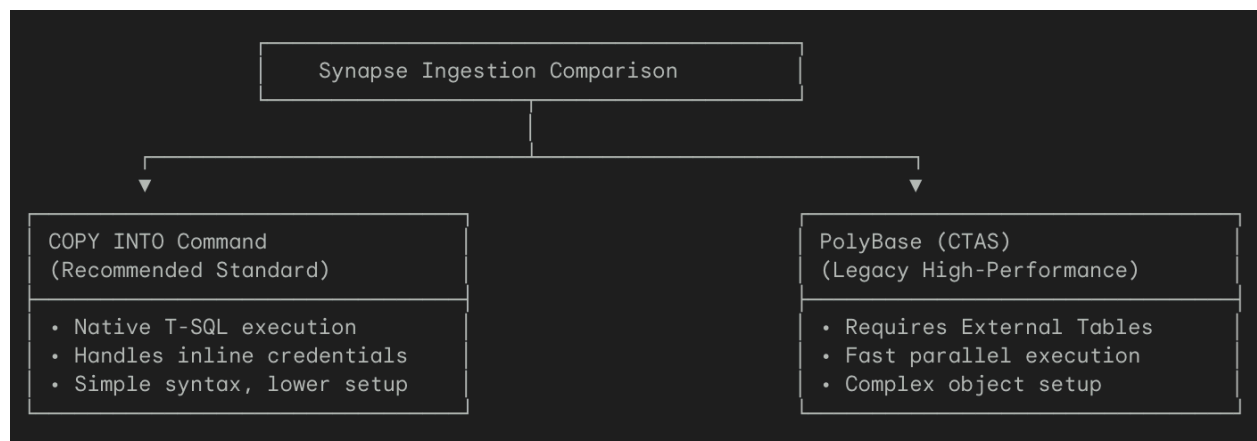
Creating Views in Serverless SQL Pools

Logical **Views** provide simplified abstraction layers for business reporting tools without generating duplicate data files in storage.

```
CREATE VIEW dbo.vw_DailySalesReport
AS
SELECT
    CAST(OrderDate AS DATE) AS SalesDate,
    COUNT(SalesOrderID) AS DailyOrders,
    SUM(TotalDue) AS DailyRevenue
FROM
    dbo.ExtSalesGold
GROUP BY
    CAST(OrderDate AS DATE);
```

6. High-Performance Bulk Data Loading Strategies

When populating Dedicated SQL Pools with bulk data, selecting the right ingestion mechanism directly impacts ingestion latency.



PolyBase vs. COPY INTO

Feature / Dimension	COPY INTO Command	PolyBase (CTAS)
Ease of Setup	High (single T-SQL statement)	Moderate (requires credentials, data sources, file formats)
External Table Required?	No (loads directly into destination table)	Yes (requires external table definition)
Authentication Flexibility	Shared Access Signatures , Storage Keys, Service Principals, Managed Identity	Service Principal, Managed Identity
Format Support	CSV, Parquet, ORC	CSV, Parquet, ORC
Throughput & Speed	Very High (Optimized native throughput)	Extremely High (Massively parallel background loader)

Option 1: Loading Data using COPY INTO

The COPY INTO statement provides the standard mechanism for high-throughput data loading into Dedicated SQL Pools without managing external table prerequisites.

```
COPY INTO dbo.FactSales
(SalesOrderNumber 1, CustomerKey 2, OrderDateKey 3, TotalAmount 4)
FROM 'https://stsynapseadls.dfs.core.windows.net/silver/sales_2026.parquet'
WITH
(
    FILE_TYPE = 'PARQUET',
    CREDENTIAL = (IDENTITY = 'Managed Identity'),
    AUTO_CREATE_TABLE = 'OFF'
);
```

Option 2: PolyBase Data Ingestion (CTAS)

PolyBase uses the distributed compute nodes of Synapse to ingest data in parallel from external tables. Using CTAS (**Create Table As Select**) ingests data into managed relational distributions in a single step.

-- PolyBase bulk load using CTAS

```

CREATE TABLE dbo.FactSalesPolyBase
WITH
(
    CLUSTERED COLUMNSTORE INDEX,
    DISTRIBUTION = HASH(CustomerKey)
)
AS
SELECT
    SalesOrderID,
    CustomerKey,
    OrderDateKey,
    TotalAmount
FROM
    dbo.ExtSalesGold;

```

7. Operational Best Practices Summary

1. **Always Match Distribution Keys for Joins:** When joining two large fact tables, hash-distribute both tables on the same column (e.g., CustomerKey) to prevent Data Movement Service shuffles across compute nodes.
2. **Replicate Small Dimensions:** Mark dimension tables under 2gb as REPLICATE to eliminate query shuffling.
3. **Use COPY INTO for ETL Pipelines:** Prefer COPY INTO over traditional INSERT...SELECT statements for bulk loads, as COPY INTO runs in parallel across all compute distributions.
4. **Pause Compute When Idle:** Dedicated SQL Pools charge for compute allocation until paused. Pause dedicated instances when inactive to avoid unnecessary resource consumption.
5. **Optimize Serverless SQL Queries:** Partition files in ADLS Gen2 using logical folder paths (/year=2026/month=07/) and query them using r.filepath() to prune irrelevant folders during Serverless SQL execution.